Basics of Markov Chain Simulation [1]

Chapter 11 [2]

Basics of Markov chain simulation [1]

Many clever methods have been devised for constructing and sampling from arbitrary posterior distributions. [2]

Markov chain simulation (also called Markov chain Monte Carlo, or MCMC) is a general method based on drawing values of θ from approximate distributions and then correcting those draws to better approximate the target posterior distribution, $p(\theta|y)$. [3]

The sampling is done sequentially, with the distribution of the sampled draws depending on the last value drawn; hence, the draws form a Markov chain. 1

Note 2

As defined in probability theory, a Markov chain is a sequence of random variables $\theta_1, \theta_2, \ldots$, for which, for any t, the distribution of θ_t given all previous θ's depends only on the most recent value, θ_{t-1}. 3

The key to the method's success, however, is not the Markov property but rather that the approximate distributions are improved at each step in the simulation, in the sense of converging to the target distribution. 1

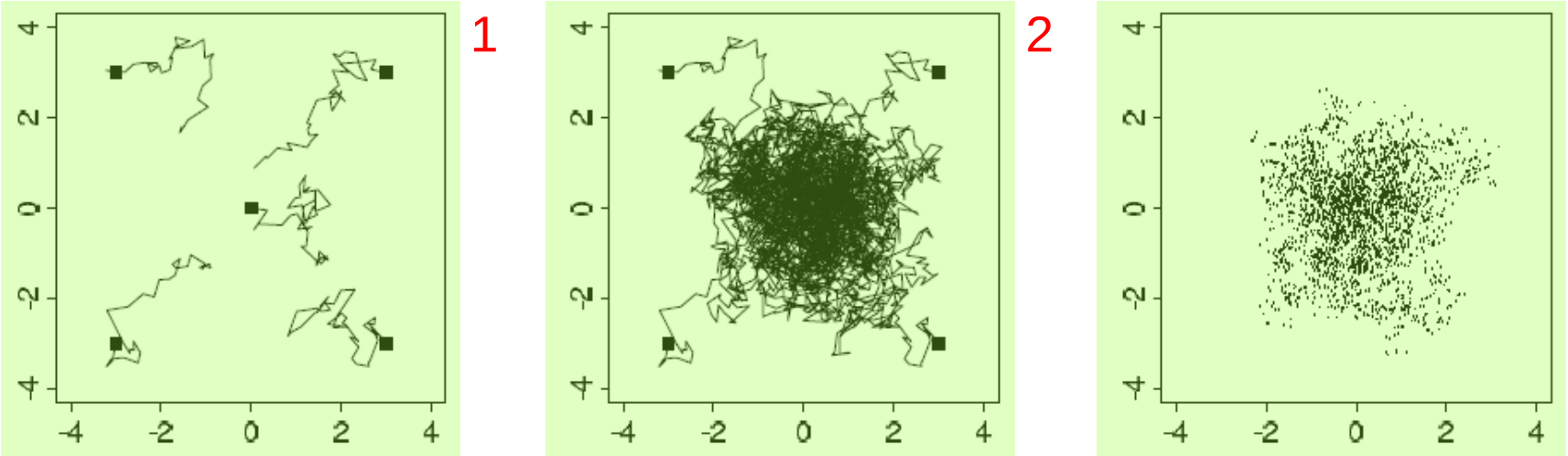


FIGURE: Five independent sequences of a Markov chain simulation for the bivariate unit normal distribution, with overdispersed starting points indicated by solid squares. The simulation is a Metropolis algorithm described in the example on page 278, with a jumping rule that has purposely been chosen to be inefficient so that the chains will move slowly and their random-walk-like aspect will be apparent. 4

Figure 11.1 illustrates a simple example of a Markov chain simulation. In this case, a Metropolis algorithm in which θ is a vector with only two components, with a bivariate unit normal posterior distribution, $\theta \sim N(0, I)$. 1

First consider Figure 11.1a, which portrays the early stages of the simulation. The space of the figure represents the range of possible values of the multivariate parameter, θ, and each of the five jagged lines represents the early path of a random walk starting near the center or the extremes of the target distribution and jumping through the distribution according to an appropriate sequence of random iterations. 2

Figure 11.1b represents the mature stage of the same Markov chain simulation, in which the simulated random walks have each traced a path throughout the space of θ, with a common stationary distribution that is equal to the target distribution. 1

We can then perform inferences about θ using points from the second halves of the Markov chains we have simulated, as displayed in Figure 11.1c. 2

In our applications of Markov chain simulation, we create several independent sequences; each sequence, $\theta^1, \theta^2, \theta^3, \ldots$, is produced by 1

- starting at some point θ^0 and then, 2
- for each t, drawing θ^t from a transition distribution, $T_t(\theta^t|\theta^{t-1})$ that depends on the previous draw, θ^{t-1}.

As we shall see in the discussion of combining the Gibbs sampler and Metropolis sampling, it is often convenient to allow the transition distribution to depend on the iteration number t; hence the notation T_t. 1

The transition probability distributions must be constructed so that the Markov chain converges to a unique stationary distribution that is the posterior distribution, $p(\theta|y)$. 2

Markov chain simulation is used when it is not possible (or not computationally efficient) to sample θ directly from $p(\theta|y)$; instead we sample iteratively in such a way that at each step of the process we expect to draw from a distribution that becomes closer to $p(\theta|y)$. 1

For a wide class of problems (including posterior distributions for many hierarchical models), this appears to be the easiest way to get reliable results. 2

The key to Markov chain simulation is to create a Markov process whose stationary distribution is the specified $p(\theta|y)$ and to run the simulation long enough that the distribution of the current draws is close enough to this stationary distribution. 1

For any specific $p(\theta|y)$, or unnormalized density $q(\theta|y)$, a variety of Markov chains with the desired property can be constructed. 2

Once the simulation algorithm has been implemented and the simulations drawn, it is absolutely necessary to check the convergence of the simulated sequences; for example, the simulations of Figure 11.1a are far from convergence and are not close to the target distribution. 1

We will discuss how to check convergence, and construct an expression for the effective number of simulation draws for a correlated sample. 2

This chapter introduces the basic Markov chain simulation methods, the Gibbs sampler and the Metropolis-Hastings algorithm, in the context of our general computing approach based on successive approximation. 1

We sketch a proof of the convergence of Markov chain simulation algorithms and present a method for monitoring the convergence in practice. 2

1 Gibbs Sampler 1

2 Metropolis and Metropolis-Hastings Algorithms 2

3 Using Gibbs and Metropolis as Building Blocks

4 Inference and Assessing Convergence

5 Effective Number of Simulation Draws

6 Example: Hierarchical Normal Model

Gibbs sampler 1

A particular Markov chain algorithm that has been found useful in many 2
multidimensional problems is the Gibbs sampler, also called alternating conditional sampling, which is defined in terms of subvectors of θ.

Suppose the parameter vector θ has been divided into d components or 3
subvectors, $\theta = (\theta_1, \ldots, \theta_d)$.

Each iteration of the Gibbs sampler cycles through the subvectors of θ, 4
drawing each subset conditional on the value of all the others. There are thus d steps in iteration t.

At each iteration t, an ordering of the d subvectors of θ is chosen and, in turn, each θ_j^t is sampled from the conditional distribution given all the other components of θ: 1

$$p(\theta_j|\theta_{-j}^{t-1}, y).$$ 2

- θ_{-j}^{t-1} represents all the components of θ, except for θ_j, at their current values: 3

$$\theta_{-j}^{t-1} = (\theta_1^t, \ldots, \theta_{j-1}^t, \theta_{j+1}^{t-1}, \ldots, \theta_d^t).$$ 4

Thus, each subvector θ_j is updated conditional on the latest values of the other components of θ, which are the iteration t values for the components already updated and the iteration $t-1$ values for the others. 1

For many problems involving standard statistical models, it is possible to sample directly from most or all of the conditional posterior distributions of the parameters. 1

We typically construct models using a sequence of conditional probability distributions, as in the hierarchical models of Chapter 5. 2

It is often the case that the conditional distributions in such models are conjugate distributions that provide for easy simulation. 3

Example: Bivariate normal distribution[1]

See the textbook.[2]

1 Gibbs Sampler

2 Metropolis and Metropolis-Hastings Algorithms

3 Using Gibbs and Metropolis as Building Blocks

4 Inference and Assessing Convergence

5 Effective Number of Simulation Draws

6 Example: Hierarchical Normal Model

1

Metropolis and Metropolis-Hastings 1

algorithms 2

The Metropolis-Hastings algorithm is a general term for a family of Markov chain simulation methods that are useful for sampling from Bayesian posterior distributions. 3

We have already seen the Gibbs sampler in the previous section; it can be viewed as a special case of Metropolis-Hastings. 4

Here we present the basic Metropolis algorithm and its generalization to the Metropolis-Hastings algorithm. 5

The Metropolis algorithm 1

The Metropolis algorithm is an adaptation of a random walk with an acceptance/rejection rule to converge to the specified target distribution. 2

The algorithm proceeds as follows. 3

1. Draw a starting point θ^0, for which $p(\theta^0|y) > 0$, from a starting distribution $p_0(\theta)$. The starting distribution might, for example, be based on an approximation as described in Section 13.3. Or we may simply choose starting values dispersed around a crude approximate estimate of the sort discussed in Chapter 10. 4

2. For $t = 1, 2, \ldots$: 1

(2.1) Sample a proposal θ^* from a jumping distribution (or proposal distribution) at time t, $J_t(\theta^* | \theta^{t-1})$. 2

For the Metropolis algorithm (but not the Metropolis-Hastings algorithm, as discussed later in this section), the jumping distribution must be symmetric, satisfying the condition $J_t(\theta_a | \theta_b) = J_t(\theta_b | \theta_a)$ for all θ_a, θ_b, and t. 3

2. For $t = 1, 2, \ldots$: 1

(2.2) Calculate the ratio of the densities, 2

$$r = \frac{p(\theta^* | y)}{p(\theta^{t-1} | y)}. \tag{11.1}$$ 3

(2.3) Set 4

$$\theta^t = \begin{cases} \theta^*, & \text{with probability } \min(r, 1), \\ \theta^{t-1}, & \text{otherwise.} \end{cases}$$ 5

Given the current value θ^{t-1}, the transition distribution $T_t(\theta^t|\theta^{t-1})$ of the Markov chain is thus a mixture of a point mass at $\theta^t = \theta^{t-1}$, and a weighted version of the jumping distribution, $J_t(\theta^t|\theta^{t-1})$, that adjusts for the acceptance rate. 1

The algorithm requires the ability to calculate the ratio r in (11.1) for all (θ, θ^*), and to draw θ from the jumping distribution $J_t(\theta^*|\theta)$ for all θ and t. 1

In addition, step (2.3) above requires the generation of a uniform random number. 2

Note 3

When $\theta^t = \theta^{t-1}$, that is, if the jump is not accepted, this still counts as an iteration in the algorithm. 4

Example: Bivariate unit normal density with normal jumping kernel [1]

See the textbook. [2]

Relation to optimization 1

The acceptance/rejection rule of the Metropolis algorithm can be stated as follows: 2

(1) if the jump increases the posterior density, set $\theta^t = \theta^*$; 3

(2) if the jump decreases the posterior density, set $\theta^t = \theta^*$ with probability equal to the density ratio, r, and set $\theta^t = \theta^{t-1}$ otherwise.

The Metropolis algorithm can thus be viewed as a stochastic version of a stepwise mode-finding algorithm, always accepting steps that increase the density but only sometimes accepting downward steps. 1

Why does the Metropolis algorithm work?[1]

The proof that the sequence of iterations $\theta^1, \theta^2, \ldots$ converges to the target distribution has two steps:[2]

- first, it is shown that the simulated sequence is a Markov chain with a unique stationary distribution,[3]
- second, it is shown that the stationary distribution equals this target distribution.

The first step of the proof holds if the Markov chain is irreducible, aperiodic, and not transient. 1

Except for trivial exceptions, the latter two conditions hold for a random walk on any proper distribution, and irreducibility holds as long as the random walk has a positive probability of eventually reaching any state from any other state; that is, the jumping distributions J_t must eventually be able to jump to all states with positive probability. 2

To see that the target distribution is the stationary distribution of the Markov chain generated by the Metropolis algorithm, consider starting the algorithm at time $t-1$ with a draw θ^{t-1} from the target distribution $p(\theta|y)$. 1

Now consider any two such points θ_a and θ_b, drawn from $p(\theta|y)$ and labeled so that $p(\theta_b|y) \geq p(\theta_a|y)$. 2

The unconditional probability density of a transition from θ_a and θ_b is 1

$$p(\theta^{t-1} = \theta_a, \theta^t = \theta_b) = p(\theta_a|y) \times J_t(\theta_b|\theta_a), \quad 2$$

where the acceptance probability is 1 because of our labeling of a and b, 3 and the unconditional probability density of a transition from θ_b and θ_a is, from (11.1),

$$\begin{aligned} p(\theta^t = \theta_a, \theta^{t-1} = \theta_b) &= p(\theta_b|y) \times J_t(\theta_a|\theta_b) \times \frac{p(\theta_a|y)}{p(\theta_b|y)} \quad 4 \\ &= p(\theta_a|y) \times J_t(\theta_a|\theta_b), \end{aligned}$$

where is the same as the probability of a transition from θ_a to θ_b, since we 5 have required that $J_t(.|.)$ be symmetric, that is, $J_t(\theta_a|\theta_b) \equiv J_t(\theta_b|\theta_a)$.

Since their joint distribution is symmetric, θ^t and θ^{t-1} have the same marginal distributions, and so $p(\theta|y)$ is the stationary distribution of the Markov chain of θ. 1

The Metropolis-Hastings algorithm 1

The Metropolis-Hastings algorithm generalizes the basic Metropolis algorithm presented above in two ways. 2

First, the jumping rules J_t need no longer be symmetric; that is, there is no requirement that $J_t(\theta_a|\theta_b) \equiv J_t(\theta_b|\theta_a)$. 3

Second, to correct for the asymmetry in the jumping rule, the ratio r in (11.1) is replaced by a ratio of ratios: 1

$$r = \frac{p(\theta^*|y)}{p(\theta^{t-1}|y)} \times \frac{J_t(\theta^{t-1}|\theta^*)}{J_t(\theta^*|\theta^{t-1})}. \tag{11.2}$$ 2

Note 3

The ratio r is always defined, because a jump from θ^{t-1} to θ^* can only occur if both $p(\theta^{t-1}|y)$ and $J_t(\theta^*|\theta^{t-1})$ are nonzero. 4

Allowing asymmetric jumping rules can be useful in increasing the speed of the random walk. 1

Convergence to the target distribution is proved in the same way as for the Metropolis algorithm. 2

The proof of convergence to a unique stationary distribution is identical. 3

To prove that the stationary distribution is the target distribution, $p(\theta|y)$, consider any two points θ_a and θ_b with posterior densities labeled so that $p(\theta_b|y)J_t(\theta_a|\theta_b) \geq p(\theta_a|y)J_t(\theta_b|\theta_a)$. 1

If θ^{t-1} follows the target distribution, then it is easy to show that the unconditional probability density of a transition from θ_a to θ_b is the same as the reverse transition. 2

Relation between the jumping rule and efficiency of simulations 1

The ideal Metropolis-Hastings jumping rule is simply to sample the proposal, 2
θ^*, from the target distribution; that is, $J(\theta^*|\theta) \equiv p(\theta^*|y)$ for all θ.

Then the ratio r in (11.2) is always exactly 1, and the iterates θ^t are a 3
sequence of independent draws from $p(\theta|y)$.

In general, however, iterative simulation is applied to problems for which 4
direct sampling is not possible.

A good jumping distribution has the following properties: 1

- For any θ, it is easy to sample from $J(\theta^*|\theta)$. 2
- It is easy to compute the ratio r.
- Each jump goes a reasonable distance in the parameter space (otherwise the random walk moves too slowly).
- The jumps are not rejected too frequently (otherwise the random walk wastes too much time standing still).

1 Gibbs Sampler

2 Metropolis and Metropolis-Hastings Algorithms

3 Using Gibbs and Metropolis as Building Blocks

4 Inference and Assessing Convergence

5 Effective Number of Simulation Draws

6 Example: Hierarchical Normal Model

1

Using Gibbs and Metropolis as building blocks 1

The Gibbs sampler and the Metropolis algorithm can be used in various combinations to sample from complicated distributions. 2

The Gibbs sampler is the simplest of the Markov chain simulation algorithms, and it is our first choice for conditionally conjugate models, where we can directly sample from each conditional posterior distribution. 3

For example, we could use the Gibbs sampler for the normal-normal hierarchical models in Chapter 5. 4

The Metropolis algorithm can be used for models that are not conditionally conjugate, for example, the two-parameter logistic regression for the bioassay experiment in Section 3.7. 1

In this example, the Metropolis algorithm could be performed in vector form, jumping in the two-dimensional space of (α, β), or embedded within a Gibbs sampler structure, by alternately updating α and β using one-dimensional Metropolis jumps. 2

In either case, the Metropolis algorithm will probably have to be tuned to get a good acceptance rate, as discussed in Section 12.2. 3

If some of the conditional posterior distributions in a model can be sampled from directly and some cannot, then the parameters can be updated one at a time, with the Gibbs sampler used where possible and one-dimensional Metropolis updating used otherwise. 1

More generally, the parameters can be updated in blocks, where each block is altered using the Gibbs sampler or a Metropolis jump of the parameters within the block. 2

Interpretation of the Gibbs sampler as a special case of the Metropolis-Hastings algorithm 1

Gibbs sampling can be viewed as a special case of the Metropolis-Hastings algorithm in the following way. 2

We first define iteration t to consist of a series of d steps, with step j of iteration t corresponding to an update of the subvector θ_j conditional on all the other elements of θ. 3

Then the jumping distribution, $J_{j,t}(.|.)$, at step j of iteration t only jumps along the jth subvector, and does so with the conditional posterior density of θ_j given θ_{-j}^{t-1}: 1

$$J_{j,t}^{\text{Gibbs}}(\theta^*|\theta^{t-1}) = \begin{cases} p(\theta_j^*|\theta_{-j}^{t-1}, y), & \text{if } \theta_{-j}^* = \theta_{-j}^{t-1}, \\ \\ 0, & \text{otherwise.} \end{cases}$$ 2

The only possible jumps are to parameter vectors θ^* that match θ^{t-1} on all components other than the jth. 3

Under this jumping distribution, the ratio (11.2) at the jth step of iteration t is 1

$$r = \frac{p(\theta^*|y)}{p(\theta^{t-1}|y)} \times \frac{J_{j,t}^{\text{Gibbs}}(\theta^{t-1}|\theta^*)}{J_{j,t}^{\text{Gibbs}}(\theta^*|\theta^{t-1})} = \frac{p(\theta^*|y)}{p(\theta^{t-1}|y)} \times \frac{p(\theta_j^{t-1}|\theta_{-j}^{t-1}, y)}{p(\theta_j^*|\theta_{-j}^{t-1}, y)}$$ 2

$$= \frac{p(\theta_j^*|\theta_{-j}^*, y) \times p(\theta_{-j}^*|y)}{p(\theta_j^{t-1}|\theta_{-j}^{t-1}, y) \times p(\theta_{-j}^{t-1}|y)} \times \frac{p(\theta_j^{t-1}|\theta_{-j}^{t-1}, y)}{p(\theta_j^*|\theta_{-j}^{t-1}, y)} \qquad (\theta_{-j}^* = \theta_{-j}^{t-1})$$

$$= \frac{p(\theta_{-j}^{t-1}|y)}{p(\theta_{-j}^{t-1}|y)} \equiv 1,$$

and thus every jump is accepted. 3

The second line above follows from the first because, under this jumping rule, θ^* differs from θ^{t-1} only in the jth component. 1

The third line follows from the second by applying the rules of conditional probability to $\theta = (\theta_j, \theta_{-j})$ and noting that $\theta^*_{-j} = \theta^{t-1}_{-j}$. 2

Usually, one iteration of the Gibbs sampler is defined as we do, to include all d steps corresponding to the d components of θ, thereby updating all of θ at each iteration. 1

It is possible, however, to define Gibbs sampling without the restriction that each component be updated in each iteration, as long as each component is updated periodically. 2

Gibbs sampler with approximations 1

For some problems, sampling from some, or all, of the conditional distributions $p(\theta_j|\theta_{-j,y})$ is impossible, but one can construct approximations, which we label $g(\theta_j|\theta_{-j})$, from which sampling is possible. 2

The general form of the Metropolis-Hastings algorithm can be used to compensate for the approximation. 1

As in the Gibbs sampler, we choose an order for altering the d elements of θ; the jumping function at the jth Metropolis step at iteration t is then 2

$$J_{j,t}(\theta^*|\theta^{t-1}) = \begin{cases} g(\theta_j^*|\theta_{-j}^{t-1}), & \text{if } \theta_{-j}^* = \theta_{-j}^{t-1}, \\ \\ 0, & \text{otherwise,} \end{cases}$$ 3

and the ratio r in (11.2) must be computed and the acceptance or rejection of θ^* decided. 4

1 Gibbs Sampler

2 Metropolis and Metropolis-Hastings Algorithms

3 Using Gibbs and Metropolis as Building Blocks

4 Inference and Assessing Convergence

5 Effective Number of Simulation Draws

6 Example: Hierarchical Normal Model

1

Inference and assessing convergence 1

The basic method of inference from iterative simulation is the same as for Bayesian simulation in general: use the collection of all the simulated draws from $p(\theta|y)$ to summarize the posterior density and to compute quantiles, moments, and other summaries of interest as needed. 2

Posterior predictive simulations of unobserved outcomes $\tilde{y}$ can be obtained by simulation conditional on the drawn values of θ. 3

Inference using the iterative simulation draws requires some care, however, as we discuss in this section. 4

Difficulties of inference from iterative simulation 1

Iterative simulation adds two challenges to simulation inference. 2

First, if the iterations have not proceeded long enough, as in Figure 11.1a, the simulations may be grossly unrepresentative of the target distribution. 3

Even when simulations have reached approximate convergence, early iterations still reflect the starting approximation rather than the target distribution; for example, consider the early iterations of Figures 11.1b and 11.2b. 4

The second problem with iterative simulation draws is their within-sequence correlation; aside from any convergence issues, simulation inference from correlated draws is generally less precise than from the same number of independent draws. 1

Serial correlation in the simulations is not necessarily a problem because, at convergence, the draws are identically distributed as $p(\theta|y)$, and so when performing inferences, we ignore the order of the simulation draws in any case. 2

But such correlation can cause inefficiencies in simulations. 1

Consider Figure 11.1c, which displays 500 successive iterations from each of five simulated sequences of the Metropolis algorithm: the patchy appearance of the scatterplot would not be likely to appear from 2500 independent draws from the normal distribution but is rather a result of the slow movement of the simulation algorithm. 2

In some sense, the effective number of simulation draws here is far fewer than 2500. We calculate effective sample size using formula (11.8). 3

We handle the special problems of iterative simulation in three ways. 1

First, we attempt to design the simulation runs to allow effective monitoring of convergence, in particular by simulating multiple sequences with starting points dispersed throughout parameter space, as in Figure 11.1a. 2

Second, we monitor the convergence of all quantities of interest by comparing variation between and within simulated sequences until within variation roughly equals between variation, as in Figure 11.1b. 1

Only when the distribution of each simulated sequence is close to the distribution of all the sequences mixed together can they all be approximating the target distribution. 2

Third, if the simulation efficiency is unacceptably low (in the sense of requiring too much real time on the computer to obtain approximate convergence of posterior inferences for quantities of interest), the algorithm can be altered, as we discuss in Sections 12.1 and 12.2. 1

Discarding early iterations of the simulation runs 1

To diminish the influence of the starting values, we generally discard the first half of each sequence and focus attention on the second half. 2

Our inferences will be based on the assumption that the distributions of the simulated values θ^t, for large enough t, are close to the target distribution, $p(\theta|y)$. 3

We refer to the practice of discarding early iterations in Markov chain simulation as warm-up; depending on the context, different warm-up fractions can be appropriate. 4

For example, in the Gibbs sampler displayed in Figure 11.2, it would be necessary to discard only a few initial iterations. 1

We adopt the general practice of discarding the first half as a conservative choice. 2

For example, we might run 200 iterations and discard the first half. If approximate convergence has not yet been reached, we might then run another 200 iterations, now discarding all of the initial 200 iterations. 3

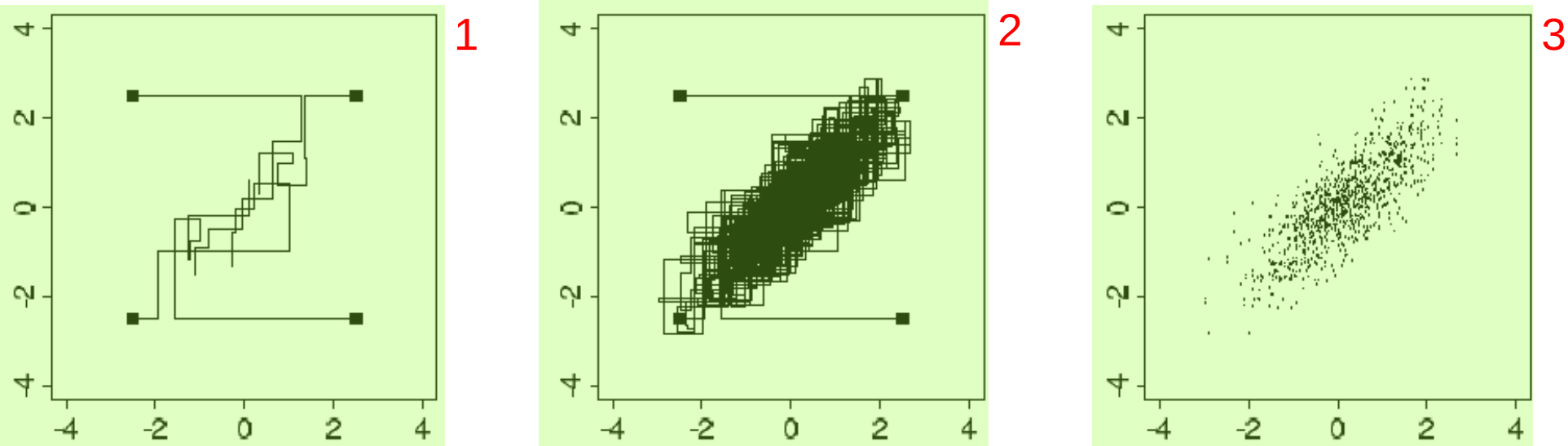


FIGURE: Four independent sequences of the Gibbs sampler for a bivariate normal distribution with correlation $\rho = 0.8$, with overdispersed starting points indicated by solid squares.

Note[1]

(A) First 10 iterations, showing the componentwise updating of the Gibbs[2] iterations.

(B) After 500 iterations, the sequences have reached approximate convergence.

(C) This figure shows the points from the second halves of the sequences, representing a set of correlated draws from the target distribution.

Dependence of the iterations in each sequence 1

Another issue that sometimes arises, once approximate convergence has been reached, is whether to thin the sequences by keeping every kth simulation draw from each sequence and discarding the rest. 2

In our applications, we have found it useful to skip iterations in problems with large numbers of parameters where computer storage is a problem, perhaps setting k so that the total number of iterations saved is no more than 1000. 3

Note 1

Whether or not the sequences are thinned, if the sequences have reached approximate convergence, they can be directly used for inferences about the parameters θ and any other quantities of interest. 2

Multiple sequences with overdispersed[1] starting points[2]

Our recommended approach to assessing convergence of iterative simulation[3] is based on comparing different simulated sequences, as illustrated in Figure 11.1, which shows five parallel simulations before and after approximate convergence.

In Figure 11.1a, the multiple sequences clearly have not converged; the variance within each sequence is much less than the variance between sequences. 1

Later, in Figure 11.1b, the sequences have mixed, and the two variance components are essentially equal. 2

To see such disparities, we clearly need more than one independent sequence. 1

Thus our plan is to simulate independently at least two sequences, with starting points drawn from an overdispersed distribution. 2

Monitoring scalar estimands 1

We monitor each scalar estimand or other scalar quantities of interest separately. 2

Estimands include all the parameters of interest in the model and any other quantities of interest (for example, the ratio of two parameters or the value of a predicted future observation). 3

It is often useful also to monitor the value of the logarithm of the posterior density, which has probably already been computed if we are using a version of the Metropolis algorithm. 4

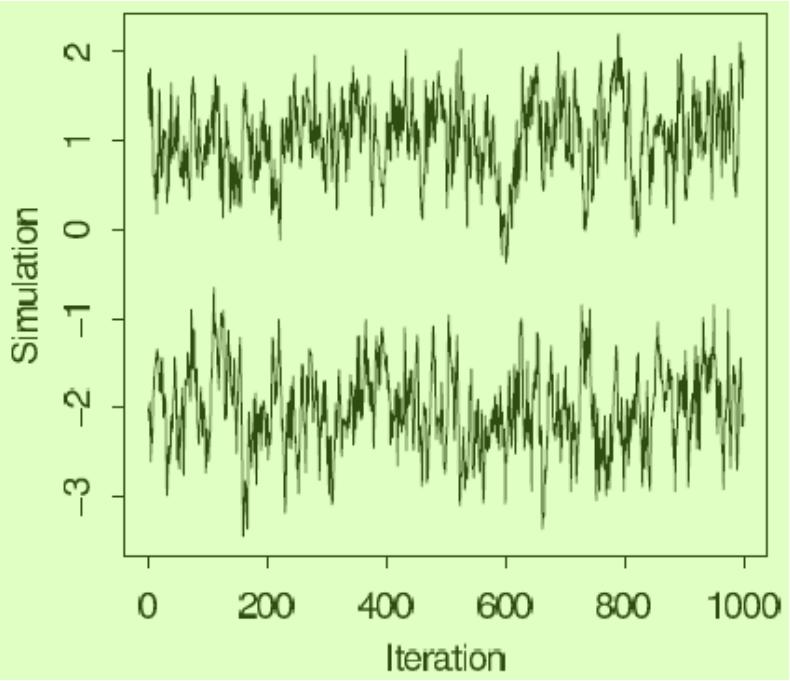
Challenges of monitoring convergence: 1

mixing and stationarity 2

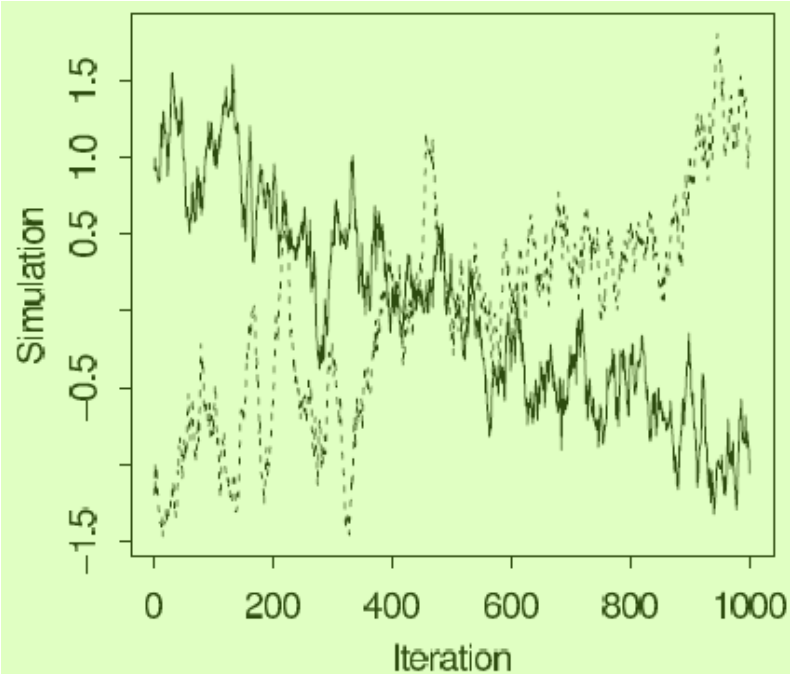
Figure 11.3 illustrates two of the challenges of monitoring convergence of iterative simulations. 3

The first graph shows two sequences, each of which looks fine on its own (and, indeed, when looked at separately would satisfy any reasonable convergence criterion), but when looked at together reveal a clear lack of convergence. 4

Figure 11.3a illustrates that, to achieve convergence, the sequences must together have mixed. 5



1



2

FIGURE: Examples of two challenges in assessing convergence of iterative simulations. 3

Note[1]

(1) In the left plot, either sequence alone looks stable, but the juxtaposition[2] makes it clear that they have not converged to a common distribution.

(2) In the right plot, the two sequences happen to cover a common distribution but neither sequence appears stationary.

(3) These graphs demonstrate the need to use between-sequence and also within-sequence information when assessing convergence.

The second graph in Figure 11.3 shows two chains that have mixed, in the sense that they have traced out a common distribution, but they do not appear to have converged. 1

Figure 11.3b illustrates that, to achieve convergence, each individual sequence must reach stationarity. 2

Splitting each saved sequence into two parts 1

We diagnose convergence (as noted above, separately for each scalar quantity of interest) by checking mixing and stationarity. 2

There are various ways to do this; we apply a fairly simple approach in which we split each chain in half and check that all the resulting half-sequences have mixed. 3

This simultaneously tests 1

- mixing: if all the chains have mixed well, the separate parts of the different chains should also mix; 2
- stationarity: at stationarity, the first and second half of each sequence should be traversing the same distribution.

We start with some number of simulated sequences in which the warm-up period (which by default we set to the first half of the simulations) has already been discarded. 1

We then take each of these chains and split into the first and second half (this is all after discarding the warm-up iterations). 2

Let m be the number of chains (after splitting) and n be the length of each chain. 1

We always simulate at least two sequences so that we can observe mixing; see Figure 11.3a; thus m is always at least 4. 2

For example, suppose we simulate 5 chains, each of length 1000, and then discard the first half of each as warm-up. 1

We are then left with 5 chains, each of length 500, and we split each into two parts: iterations $1 \sim 250$ (originally iterations $501 \sim 750$) and iterations $251 \sim 500$ (originally iterations $751 \sim 1000$). 2

We now have $m = 10$ chains, each of length $n = 250$. 3

Assessing mixing using between- and within-sequence variances 1

For each scalar estimand ψ, we label the simulations as ψ_{ij}, $i = 1, \ldots, n$; 2
$j = 1, \ldots, m$, and we compute B and W, the between- and within-sequence variances:

$$B = \frac{n}{m-1} \sum_{j=1}^{m} (\bar{\psi}_{.j} - \bar{\psi}_{..})^2, \quad \bar{\psi}_{.j} = \frac{1}{n} \sum_{i=1}^{n} \psi_{ij}, \quad \bar{\psi}_{..} = \frac{1}{m} \sum_{j=1}^{m} \bar{\psi}_{.j}, \quad 3$$

$$W = \frac{1}{m} \sum_{j=1}^{m} s_j^2, \quad s_j^2 = \frac{1}{n-1} \sum_{i=1}^{n} (\psi_{ij} - \bar{\psi}_{.j})^2. \quad 4$$

The between-sequence variance, B, contains a factor of n because it is based on the variance of the within-sequence means, $\bar{\psi}_{.j}$, each of which is an average of n values ψ_{ij}. 1

We can estimate $\mathrm{Var}(\psi|y)$, the marginal posterior variance of the estimand, by a weighted average of W and B, namely 2

$$\widehat{\mathrm{Var}}^{+}(\psi|y) = \frac{n-1}{n}W + \frac{1}{n}B. \qquad (11.3)$$ 3

This quantity overestimates the marginal posterior variance assuming the starting distribution is appropriately overdispersed, but is unbiased under stationarity (that is, if the starting distribution equals the target distribution), or in the limit $n \to \infty$. 1

This is analogous to the classical variance estimate with cluster sampling. 2

Meanwhile, for any finite n, the within variance W should be an underestimate of $\mathrm{Var}(\psi|y)$ because the individual sequences have not had time to range over all of the target distribution and, as a result, will have less variability; in the limit as $n \to \infty$, the expectation of W approaches $\mathrm{Var}(\psi|y)$. 1

We monitor convergence of the iterative simulation by estimating the factor by which the scale of the current distribution for ψ might be reduced if the simulations were continued in the limit $n \to \infty$. 2

This potential scale reduction is estimated by 1

$$\hat{R} = \sqrt{\frac{\widehat{\mathrm{Var}}^{+}(\psi|y)}{W}}, \tag{11.4}$$ 2

which declines to 1 as $n \to \infty$. 3

If the potential scale reduction is high, then we have reason to believe that 4 proceeding with further simulations may improve our inference about the target distribution of the associated scalar estimand.

Example: Bivariate unit normal density with normal jumping kernel (cont.) [1]

See the textbook. [2]

The method of monitoring convergence presented here has the key advantage of not requiring the user to examine time series graphs of simulated sequences. 1

Inspection of such plots is a notoriously unreliable method of assessing convergence and in addition is unwieldy when monitoring a large number of quantities of interest, such as can arise in complicated hierarchical models. 2

Because it is based on means and variances, the simple method presented here is most effective for quantities whose marginal posterior distributions are approximately normal. 1

When performing inference for extreme quantiles, or for parameters with multimodal marginal posterior distributions, one should monitor also extreme quantiles of the between and within sequences. 2

1 Gibbs Sampler

2 Metropolis and Metropolis-Hastings Algorithms

3 Using Gibbs and Metropolis as Building Blocks

4 Inference and Assessing Convergence

5 Effective Number of Simulation Draws

6 Example: Hierarchical Normal Model

1

Effective number of simulation draws[1]

Once the simulated sequences have mixed, we can compute an approximate effective number of independent simulation draws for any estimand of interest ψ. [2]

We start with the observation that if the n simulation draws within each sequence were truly independent, then the between-sequence variance B would be an unbiased estimate of the posterior variance, $\text{Var}(\psi|y)$, and we would have a total of mn independent simulations from the m sequences. [3]

In general, however, the simulations of ψ within each sequence will be autocorrelated, and B will be larger than $\mathrm{Var}(\psi|y)$, in expectation. 1

One way to define effective sample size for correlated simulation draws is to consider the statistical efficiency of the average of the simulations $\bar{\psi}_{..}$, as an estimate of the posterior mean, $\mathrm{E}(\psi|y)$. 2

This can be a reasonable baseline even though is not the only possible summary and might be inappropriate, for example, if there is particular interest in accurate representation of low-probability events in the tails of the distribution. 3

Continuing with this definition, it is usual to compute effective sample size using the following asymptotic formula for the variance of the average of a correlated sequence: 1

$$\lim_{n\to\infty} mn\mathrm{Var}(\bar{\psi}_{..}) = \left(1 + 2\sum_{t=1}^{\infty}\rho_t\right)\mathrm{Var}(\psi|y). \qquad (11.5)$$ 2

ρ_t: the autocorrelation of the sequence ψ at lag t. 3

If the n simulation draws from each of the m chains were independent, then 1
$\mathrm{Var}(\bar{\psi}_{..})$ would simply be $\frac{1}{mn}\mathrm{Var}(\psi|y)$ and the sample size would be mn.

In the presence of correlation we then define the effective sample size as 2

$$n_{\text{eff}} = \frac{mn}{1 + 2\sum_{t=1}^{\infty} \rho_t}. \tag{11.6}$$ 3

The asymptotic nature of (11.5) and (11.6) might seem disturbing given that in reality we will only have a finite simulation, but this should not be a problem given that we already want to run the simulations long enough for approximate convergence to the (asymptotic) target distribution. 1

To compute the effective sample size we need an estimate of the sum of the correlations ρ, for which we use information within and between sequences. 1

We start by computing the total variance using the estimate $\widehat{\mathrm{Var}}^{+}$ from (11.3); we then estimate the correlations by first computing the variogram V_t at each lag t: 2

$$V_t = \frac{1}{m(n-t)} \sum_{j=1}^{m} \sum_{i=t+1}^{n} (\psi_{i,j} - \psi_{i-t,j})^2. \quad 3$$

We then estimate the correlations by inverting the formula, $\mathrm{E}(\psi_i - \psi_{i-t})^2 = 2(1-\rho_t)\mathrm{Var}(\psi)$: 1

$$\hat{\rho}_t = 1 - \frac{V_t}{2\widehat{\mathrm{Var}}^+}. \tag{11.7}$$ 2

Unfortunately we cannot simply sum all of these to estimate n_{eff} in (11.6); the difficulty is that for large values of t the sample correlation is too noisy. 3

Instead we compute a partial sum, starting from lag 0 and continuing until the sum of autocorrelation estimates for two successive lags $\hat{\rho}_{2t'} + \hat{\rho}_{2t'+1}$ is negative. 4

We use this positive partial sum as our estimate of $\sum_{t=1}^{\infty} \rho_t$ in (11.6). 1

Putting this all together yields the estimate, 2

$$\hat{n}_{\text{eff}} = \frac{mn}{1 + 2\sum_{t=1}^{T} \hat{\rho}_t}. \tag{11.8}$$ 3

- the estimated autocorrelations $\hat{\rho}_t$ are computed from formula (11.7) and T is the first odd positive integer for which $\hat{\rho}_{T+1} + \hat{\rho}_{T+2}$ is negative. 4

All these calculations should be performed using only the saved iterations, after discarding the warm-up period. 1

For example, suppose we simulate 4 chains, each of length 1000, and then discard the first half of each as warm-up. 2

Then $m = 8$, $n = 250$, and we compute variograms and correlations only for the saved iterations (thus, up to a maximum lag t of 249, although in practice the stopping point T in (11.8) will be much lower). 3

Bounded or long-tailed distributions 1

The above convergence diagnostics are based on means and variances, and they will not work so well for parameters or scalar summaries for which the posterior distribution, $p(\phi|y)$, is far from Gaussian. 2

As discussed in Chapter 4, asymptotically the posterior distribution should typically be normally distributed as the data sample size approaches infinity, but 1

(A) we are never actually at the asymptotic limit (in fact we are often interested in learning from small samples), and 2

(B) it is common to have only a small amount of data on individual parameters that are part of a hierarchical model.

For summaries ϕ whose distributions are constrained or otherwise far from normal, we can preprocess simulations using transformations before computing the potential scale reduction factor $\hat{R}$ and the effective sample size $\hat{n}_{\text{eff}}$. 1

We can take the logarithm of all-positive quantities, the logit of quantities that are constrained to fall in $(0, 1)$, and use the rank transformation for long-tailed distributions. 2

Transforming the simulations to have well-behaved distributions should allow mean and variance-based convergence diagnostics to work better. 3

Stopping the simulations 1

We monitor convergence for the entire multivariate distribution, $p(\theta|y)$, by computing the potential scale reduction factor (11.4) and the effective sample size (11.8) for each scalar summary of interest. 2

Recall that we are using θ to denote the vector of unknowns in the posterior distribution, and ϕ to represent scalar summaries, considered one at a time. 3

We recommend computing the potential scale reduction for all scalar estimands of interest; if $\hat{R}$ is not near 1 for all of them, continue the simulation runs (perhaps altering the simulation algorithm itself to make the simulations more efficient, as described in the next section). 1

The condition of $\hat{R}$ being near 1 depends on the problem at hand, but we generally have been satisfied with setting 1.1 as a threshold. 2

We can use effective sample size $\hat{n}_{\text{eff}}$ to give us a sense of the precision obtained from our simulations. 1

As we have discussed in Section 10.5, for many purposes it should suffice to have 100 or even 10 independent simulation draws. 2

Note 3

If $\hat{n}_{\text{eff}} = 10$, the simulation standard error is increased by $\sqrt{1 + 1/10} = 1.05$. 4

As a default rule, we suggest running the simulation until $\hat{n}_{\text{eff}}$ is at least $5m$, 1
that is, until there are the equivalent of at least 10 independent draws per sequence.

Note 2

Recall that m is twice the number of sequences, as we have split each 3
sequence into two parts so that $\hat{R}$ can assess stationarity as well as mixing.

Having an effective sample size of 10 per sequence should typically correspond to stability of all the simulated sequences. 1

For some purposes, more precision will be desired, and then a higher effective sample size threshold can be used. 2

Once $\hat{R}$ is near 1 and $\hat{n}_{\text{eff}}$ is more than 10 per chain for all scalar estimands of interest, just collect the mn simulations (with warm-up iterations already excluded, as noted before) and treat them as a sample from the target distribution. 1

Even if an iterative simulation appears to converge and has passed all tests of convergence, it still may actually be far from convergence if important areas of the target distribution were not captured by the starting distribution and are not easily reachable by the simulation algorithm. 2

When we declare approximate convergence, we are actually concluding that each individual sequence appears stationary and that the observed sequences have mixed well with each other. 1

These checks are not hypothesis tests. There is no p-value and no statistical significance. 2

We assess discrepancy from convergence via practical significance (or some conventional version thereof, such as $\hat{R} > 1.1$). 3

1 Gibbs Sampler

2 Metropolis and Metropolis-Hastings Algorithms

3 Using Gibbs and Metropolis as Building Blocks

4 Inference and Assessing Convergence

5 Effective Number of Simulation Draws

6 Example: Hierarchical Normal Model

1

Example: Hierarchical Normal Model[1]

See the textbook.[2]